

WebGameBench: Requirement-to-Application Evaluation for Coding Agents via Browser-Native Games

Wenyu Zhang^{1,*} Guoliang You^{2,*} Tianlun¹ Haotian Zhao¹
 Tianshu Zhu¹ Haoran Wang¹ Xiaoxuan Tang¹ Mingyang Dai¹
 Jingnan Gu^{1,†} Daxiang Dong^{1,†} Jianmin Wu^{1,†}
¹Baidu ²University of Science and Technology of China
 {zhangwenyu08,tianlun,zhaohaotian02,zhutianshu,wanghaoran11}@baidu.com
 {tangxiaoxuan02,daimingyang,gujingnan,dongdaxiang,wujianmin}@baidu.com
 glyou@mail.ustc.edu.cn

* Equal contribution. † Corresponding authors.

Abstract

Coding agents are increasingly used as application builders, yet many evaluations still focus on source code, repository-level tests, or intermediate traces rather than the delivered application. We introduce WebGameBench, a requirement-to-application benchmark that evaluates whether coding agents can turn a frozen Structured WebGame Specification into a browser-accessible game. Browser-native games provide a compact but behavior-dense testbed: even simple games require coordinated input handling, spatial mapping, rule execution, state transitions, terminal conditions, restart behavior, and visible feedback. In WebGameBench, each generated artifact is built, served, and exposed as a browser-accessible application under a unified deployment protocol. A runtime evaluator then interacts with the delivered game in a real browser and assigns a three-way label: EXCELLENT, USABLE, or UNUSABLE. On a human-reviewed subset, the runtime label is broadly aligned with human gameplay review under the Usable-rate criterion. Across 111 tasks, 12 coding agents, and 14 evaluation configurations, WebGameBench separates current systems: the best configuration reaches a 76.9% usable rate but only a 20.2% excellent rate. This gap shows that crossing the minimum playable-delivery threshold is still far from complete requirement satisfaction. **To our knowledge, WebGameBench is the first requirement-to-application benchmark for browser-native game delivery that validates delivered-application runtime labels against independent human gameplay review under the Usable-rate criterion.**

1 Introduction

As coding agents move from local code generation toward building applications from requirements, evaluation should move beyond code correctness and ask whether a model can understand a requirement, implement a system, deploy it locally, and leave users with an operable application. Existing benchmarks cover several important slices of this capability chain: code benchmarks such as HumanEval [1] and LiveCodeBench [2] focus on function- or program-level correctness; software-engineering benchmarks such as SWE-bench [3] and Terminal-Bench [4] focus on long-horizon work in existing repositories or terminal environments; and web and computer-use benchmarks such as WebArena [5] and OSWorld [6] evaluate task completion in existing interactive environments.

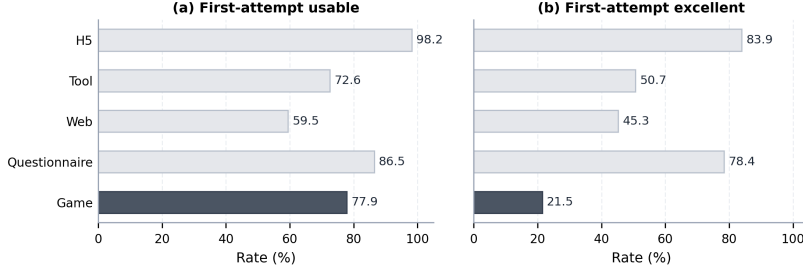


Figure 1: Pilot study on browser-native artifacts. We use ‘Opus 4.6’ to compare H5, Tool, Web, Questionnaire, and Game tasks under the same generation and evaluation process. Usable rate counts artifacts labeled EXCELLENT or USABLE; excellent rate counts artifacts labeled EXCELLENT. The result is used only to determine the benchmark scope, not for main model comparison.

However, these directions do not directly evaluate the full path from a requirement to a delivered interactive application. They typically score code, patches, terminal outcomes, or actions in existing environments, whereas a delivered application may install, build, and load while still failing to satisfy the required runtime behavior. Evaluation for this setting should therefore examine whether the application itself behaves according to the specification during real interaction.

We call this setting *requirement-to-application evaluation*. Each task provides a frozen structured requirement; the coding agent must synthesize a source artifact and deliver an application that users can operate through its interface. The evaluated unit is not an intermediate reasoning trace, isolated code fragment, repository diff, or action sequence in an existing environment, but whether the delivered application satisfies the requirement at runtime. This boundary matters because buildability and loadability are not application usability: a project may install, build, and render a page while still violating the specification in input control, spatial mapping, rule execution, state transitions, visible feedback, terminal conditions, or restart flow.

To instantiate this setting reproducibly, a benchmark needs an application substrate that reduces dependence on external services, private assets, and complex system configuration, while still supporting lightweight deployment, a unified access point, and user-level automated interaction. Browser-native web applications satisfy these requirements. In our instantiation, the generated application is deployed locally and exposed through a browser-accessible URL, which cleanly separates application synthesis from interactive runtime evaluation. This shifts assessment from source-level plausibility to the behavior of the delivered application in the browser.

Among browser-native artifacts, we use games as the primary testbed. This choice is not because game generation is the endpoint of the research problem, but because games compress dense dynamic behavior into small specifications: input response, spatial relationships, collision or hit detection, score and resource updates, state machines, win/loss conditions, restart logic, and visual feedback. These behaviors require the model not only to generate a loadable interface, but also to preserve the runtime dynamics specified by the requirement. To check that games are a practical substrate rather than only an intuitively appealing one, we conduct a pilot study. Figure 1 compares H5, Tool, Web, Questionnaire, and Game browser-native artifacts under the same generation and evaluation process. Game tasks reach a 77.9% usable rate, indicating that they are sufficiently buildable and evaluable; their excellent rate is only 21.5%, indicating that minimal usability and complete requirement satisfaction remain distinct quality levels.

Based on this testbed, we propose WebGameBench, a requirement-to-application benchmark for coding agents. WebGameBench contains 111 browser-native game tasks spanning seven gameplay families. Each task is also assigned a specification-level difficulty label $D \in \{D1, D2, D3, D4\}$, where larger D -levels indicate higher expected difficulty in generating a correct delivered application from the requirement. Each task uses a frozen Structured WebGame Specification as the sole task contract, specifying gameplay goals, interaction rules, state boundaries, deployment constraints, and requirement-derived functional points. The evaluation pipeline connects application synthesis, local deployment, and interactive runtime evaluation: the agent implements the application under a unified generation protocol; under the same generation and deployment protocol, the delivered application is exposed as a browser-accessible URL; and a runtime evaluator verifies the delivered

application through real browser interaction, producing one of three labels, EXCELLENT, USABLE, or UNUSABLE, together with structured evidence.

This design evaluates coding agents’ application-artifact delivery capability in the browser-native game setting, rather than only source-level correctness, build success, or page loading. Results on 12 coding agents and 14 evaluation configurations show interpretable model separation: the strongest configuration reaches a 76.9% usable rate but only a 20.2% excellent rate, indicating that crossing the usability threshold is not equivalent to satisfying the full requirement. Difficulty stratification also shows a marked drop in usable rate on high-risk tasks. A human-reviewed subset further shows that the runtime evaluator is reasonably aligned with human gameplay judgments under the Usable-rate criterion, while exact three-way label agreement remains substantially harder.

This paper makes three contributions. First, we formalize requirement-to-application evaluation and build WebGameBench, a closed-loop benchmark that connects specification-based generation, local deployment, and browser-interaction-based runtime evaluation for browser-native games. Second, we evaluate representative open and closed coding agents under a unified protocol and show that WebGameBench produces clear and interpretable model separation. Third, we validate the runtime evaluator against a human-reviewed subset, showing reasonable alignment with human gameplay judgments under the Usable-rate criterion while highlighting the difficulty of exact three-way quality labeling.

2 Related Work

Code and software-engineering benchmarks. Function- and program-level benchmarks established executable correctness as the core signal for code-generation evaluation. HumanEval [1], MBPP [7], APPS [8], and LiveCodeBench [2] cover function synthesis, basic program synthesis, competitive-programming tasks, and freshness / contamination-aware code evaluation. Repository-level and long-horizon engineering benchmarks such as SWE-bench [3], SWT-Bench [9], Terminal-Bench [4], SUPER [10], PaperBench [11], and MLE-bench [12] broaden evaluation to issue resolution, test generation, terminal work, research reproduction, and machine-learning engineering.

Web artifact generation and computer-use benchmarks. Web-generation benchmarks evaluate whether models can produce websites, front ends, or web applications from natural language, screenshots, sketches, or interaction prototypes [13–19]. Web and computer-use benchmarks instead evaluate agents operating existing browser, desktop, or mobile environments, including MIND2WEB [20], WebArena [5], VisualWebArena [21], WebVoyager [22], WorkArena [23], BrowserGym [24], OSWorld [6], and AndroidWorld [25].

Game-interaction benchmarks. Games are widely used for evaluating planning, exploration, spatial reasoning, and long-horizon decision making in existing environments, including text games, general video-game frameworks, Minecraft-like settings, and recent LLM/VLM game suites [26–33]. These benchmarks usually evaluate the agent as a player.

3 Method

WebGameBench instantiates *requirement-to-application evaluation* for browser-native games. Given a frozen structured requirement σ , a coding agent A makes one generation attempt in a unified environment and produces a source artifact that is deployed as a browser-native application $x = A(\sigma)$. Under the same generation and deployment protocol, the delivered application is exposed as a browser-accessible URL u for evaluation. A runtime evaluator E then judges, based on (u, σ) , whether the deployed application satisfies the requirement through browser interaction, returning a three-way quality label $y \in \{\text{EXCELLENT}, \text{USABLE}, \text{UNUSABLE}\}$ together with structured evidence. The evaluated object is therefore the observable runtime behavior of the delivered application, rather than an intermediate reasoning trace, isolated code fragment, repository diff, or action sequence in an existing environment.

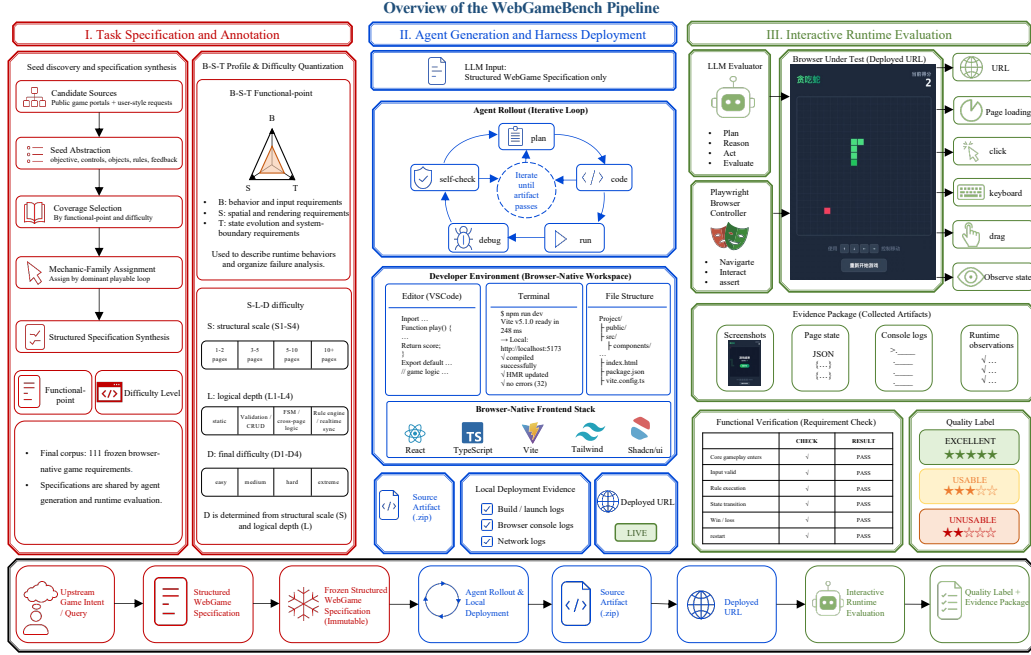


Figure 2: Overview of the WebGameBench pipeline. Each task is defined by a frozen Structured WebGame Specification; an agent produces a browser-native source artifact; the delivered application is exposed as a browser-accessible URL; and the runtime evaluator judges the delivered game against the same specification through real browser interaction.

3.1 Evaluation Setting and Protocol

Figure 2 summarizes the pipeline. The evaluation setting separates three roles that are often conflated in coding-agent benchmarks: task definition, application delivery, and runtime judgment. The Structured WebGame Specification fixes the requirement to be implemented; the coding agent attempts to deliver a browser-native application under a unified generation and deployment protocol; and the runtime evaluator judges the exposed URL against the same frozen requirement. This separation keeps the benchmark outcome tied to observable runtime behavior rather than to code appearance, build success alone, or evaluator-specific reinterpretation of the task.

Task specification and annotation. Each benchmark instance uses a frozen *Structured WebGame Specification* as its formal task input, denoted by σ . The specification follows a product-requirements-style organization while constraining the scope to browser-native game applications: it states the game objective, page or scene flow, input controls, game objects, rules and state transitions, visible feedback, deployment constraints, and observable acceptance criteria. Unlike the raw user request, σ is the shared fixed requirement representation used in evaluation: the coding agent relies on it to produce the delivered application, and the runtime evaluator judges the application against the same specification. During dataset construction, we also derive functional-point annotations and specification-level difficulty labels from σ ; both are defined in Section 3.2.

Agent generation and deployment protocol. For each task and coding agent, WebGameBench runs one independent generation attempt in a standardized browser-native development environment. The agent receives the frozen specification σ under the same workspace template, tool interface, execution budget, and deployment protocol. The attempt produces a browser-native source artifact in a project workspace and exposes the delivered application as a browser-accessible URL u . The generation record retains the source path, deployed URL, generation metadata, and trajectory log, so each delivered artifact can be traced back to the specification and agent attempt that produced it.

Interactive runtime evaluation. Interactive runtime evaluation takes the browser-accessible URL u and the same frozen specification σ as input, and asks whether the delivered application satisfies the requirement in a real browser runtime. In our implementation, the runtime evaluator is a Codex-based agent that controls Chrome and executes browser interactions through Playwright. For the main results, we use Codex CLI 0.115.0 with the ‘gpt-5.4-agent’ backend and XHigh reasoning effort; the human-review analysis additionally compares Medium, High, and XHigh reasoning settings. Each artifact receives one evaluation rollout with a two-hour wall-clock timeout and no additional fixed browser-action step cap. The evaluator plans the verification order from σ , opens u , performs user-level actions, reads page state, records runtime evidence, and produces a structured evaluation report. The evaluator is not a free-form subjective judge: each judgment must be grounded in the specification, real browser interaction, and runtime evidence.

The evaluation first determines whether the application is accessible and can enter an interactive state corresponding to the specification. It then verifies observable requirements around the main path, input response, spatial mapping, rule execution, state transition, score or resource change, terminal behavior, and restart behavior. Each judgment must be supported by user-visible page behavior, browser interaction outcomes, or stable runtime state; source code, build logs, and internal state are diagnostic evidence only, not substitutes for observable behavior. For long-horizon, randomized, spatial, or multi-session requirements, the evaluator may construct candidate preconditions or independent sessions, but the final judgment must still rest on interaction outcomes in the real browser. Finally, the runtime evaluator maps observed behavior to three quality labels: EXCELLENT, USABLE, and UNUSABLE, corresponding respectively to complete and stable satisfaction of main requirements, a playable main path with non-core defects, and failure to complete the core playable loop or to access the delivered application. These three labels form the label-valued runtime reward used by WebGameBench; usable rate and excellent rate are aggregate statistics derived from this reward.

3.2 Dataset Construction

WebGameBench constructs open but bounded browser-native game requirements. It does not collect existing web games as targets to copy, and it does not ask agents to reproduce a specific website, asset pack, or source implementation. The construction goal is a corpus of requirement-to-application tasks whose success and failure can be observed through browser runtime interaction.

Seed discovery and specification synthesis. Candidate game intents are derived from public browser-game ecosystems and real user requests. Public portals such as 4399, Poki, and CrazyGames are used only as taxonomy inspiration for gameplay families, interaction patterns, and session structures; their assets, layouts, level designs, text, and implementations are not copied. We normalize seed coverage into seven mechanic families: Puzzle/Matching, Action/Arcade/Platform, Shooting/Combat, Racing/Sports/Skill, Strategy/Simulation/Management, Casual/Reaction/Idle, and Board/Card/Local Multiplayer.

Each seed preserves the player objective, control scheme, core game objects, central rules, and observable feedback while removing branding, commercial IP, advertising, paid resources, external leaderboards, accounts, and remote services. Seeds and necessary upstream user material are then synthesized into frozen Structured WebGame Specification documents. Each specification states the minimum playable loop: how the game starts, what inputs the player can execute, how objects respond, how score or resources change, which rules trigger state transitions, when the game terminates, and how the player restarts or continues.

Functional-point annotation. After specification normalization, we annotate each frozen Structured WebGame Specification with functional points by decomposing its observable requirements along three requirement-decomposition axes: behavior, spatial, and temporal/state. Behavior (B) covers player entry, input actions, and core interaction; spatial (S) covers position representation, coordinate mapping, boards, canvas regions, hit zones, and collision relations; and temporal/state (T) covers game entities, phases, random or hidden information, resource changes, terminal conditions, state transitions, and temporal dependencies. We organize these annotations in a four-tier B-S-T functional-point schema: Tier 1 contains the three axes B , S , and T ; Tier 2 defines stable subdimensions under each axis; Tier 3 defines controlled tags or enumerated values; and Tier 4 defines task-specific atomic functional points. An atomic functional point is an independently observable behavior or structural assertion, such as game entry, input response, spatial mapping, rule execution,

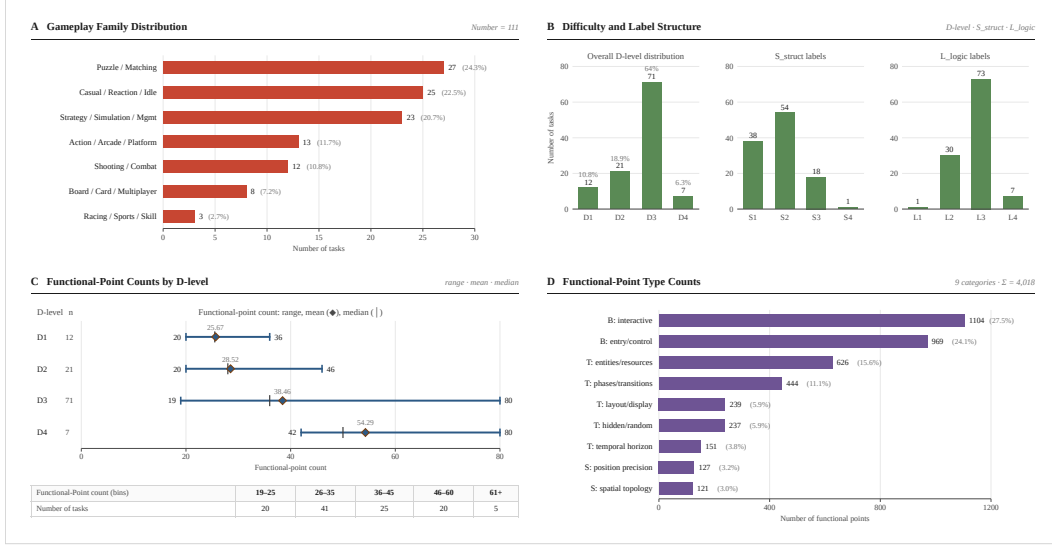


Figure 3: WebGameBench corpus profile over 111 browser-native game requirements, including gameplay families, D -level labels, S_{struct}/L_{logic} axes, and B-S-T functional-point annotations.

state transition, score or resource update, terminal triggering, or restart flow. Functional-point annotation is used only as metadata for benchmark construction, coverage analysis, and experimental stratification; the complete schema, annotation rules, and corpus statistics are given in Appendix B.

Difficulty labels. Difficulty labels characterize the expected implementation risk of a frozen Structured WebGame Specification before model evaluation. We assign this specification-level difficulty from two requirement-level axes: structural scale S_{struct} and logical depth L_{logic} . The notation S_{struct} distinguishes this difficulty axis from the spatial S in B-S-T annotation. S_{struct} measures the breadth of pages, modules, navigation hierarchy, and user-role-specific interface scope, while L_{logic} measures gameplay-rule depth, state-machine complexity, acceptance criteria, exception branches, persistence, AI behavior, and multi-session or synchronization constraints. The two axes are first assigned independently and then mapped by a fixed 4×4 lookup table to $D \in \{D1, D2, D3, D4\}$, where higher D -levels indicate higher expected implementation risk. The D -level is used for corpus profiling and difficulty-stratified analysis, not as a function of model outcomes or functional-point counts. The full annotation rubric, lookup rule, and stability analysis are given in Appendix C.

Feasibility filtering and coverage selection. After specification synthesis and task annotation, we filter candidates into the final corpus. Filtering first screens for browser-native feasibility: the core playable loop must be self-contained, and success or failure must be observable through browser runtime evidence. We then apply coverage-aware selection over three complementary dimensions: seven gameplay families for mechanic coverage, B-S-T functional-point annotation for observable requirement coverage, and D -level labels for specification-level difficulty coverage. This process is not intended to produce a proportional sample of online games, but to form a requirement-to-application benchmark corpus that is bounded, evaluable, and diverse in gameplay mechanisms, observable functional points, and expected implementation risk.

Corpus profile. Figure 3 summarizes the retained corpus. The seven gameplay families are coverage strata rather than proportional samples of online games. The corpus is structurally compact but logically dense: 82.9% of tasks have structural-scale S1 or S2 labels, while 72.1% have logical-depth L3 or L4 labels. This distribution places expected implementation risk more on gameplay rules, state transitions, scoring, terminal conditions, and multi-session behavior than on page count. The mean atomic functional-point count rises from 25.67 for D1 to 54.29 for D4, serving as a construction-time sanity check; B-S-T functional-point annotation is used for decomposition and profiling, not for assigning D -level.

Table 1: Main results on WebGameBench. Excellent rate and Usable rate use valid scored samples as the denominator; D1–D4 columns report usable/valid counts at each D -level.

Configuration	Coverage	Excellent rate	Usable rate	Usable / valid by D -level			
				D1	D2	D3	D4
opus-4-7 [34]	93.7%	20.2%	76.9%	11/12	17/21	49/65	3/6
opus-4-6 [35]	90.1%	19.0%	73.0%	9/12	17/18	45/63	2/7
gpt-5-5 [37]	99.1%	16.4%	63.6%	8/12	17/20	43/71	2/7
gemini-3.1-pro [38]	96.4%	15.9%	63.6%	10/12	15/20	42/68	1/7
deepseek-v4-flash-thinking [39]	99.1%	13.6%	62.7%	11/12	17/21	40/70	1/7
deepseek-v4-pro-thinking [39]	100.0%	12.6%	62.2%	8/12	18/21	43/71	0/7
kimi-k2.6 [40]	94.6%	16.2%	61.0%	8/12	15/21	41/65	0/7
deepseek-v4-flash-nothinking [39]	97.3%	14.8%	59.3%	11/12	17/21	35/68	1/7
GLM-5.1 [43]	97.3%	12.0%	52.8%	7/12	19/21	31/68	0/7
GLM-5 [42]	99.1%	10.9%	48.2%	8/12	14/21	31/70	0/7
sonnet-4-5 [36]	91.9%	7.8%	47.1%	7/12	15/21	25/63	1/6
deepseek-v4-pro-nothinking [39]	100.0%	16.2%	44.1%	9/12	13/21	26/71	1/7
hy3-xhigh [44]	95.5%	6.6%	41.5%	9/12	14/21	21/67	0/6
kimi-k2.5 [41]	96.4%	8.4%	38.3%	7/11	12/21	22/68	0/7

4 Experiments

4.1 Experimental Setup

All experiments use the same set of frozen Structured WebGame Specification documents. For each evaluated coding agent, WebGameBench runs the unified generation and deployment protocol defined in Section 3.1: the agent receives a frozen specification, produces a browser-native source artifact in a standardized workspace, and the artifact is built, served, and exposed as a browser-accessible URL under the same deployment protocol. During generation, all agents receive only the frozen specification and the common workspace template. The delivered application is then evaluated by the interactive runtime evaluator defined in Section 3.1. API batching concurrency, timeouts, and trace statistics are reported in Appendix E.

We report 14 evaluation configurations covering 12 representative coding agents. The count is larger than the number of agents because DeepSeek-V4 Pro and DeepSeek-V4 Flash are each evaluated under thinking and non-thinking inference settings; these rows are configuration variants within the same model family rather than additional model families. The evaluated configurations cover Anthropic Claude Opus/Sonnet models [34–36], OpenAI GPT-5.5 [37], Google Gemini 3.1 Pro [38], DeepSeek-V4 [39], Kimi K2.6 [40], Kimi K2.5 [41], GLM-5/5.1 [42, 43], and Tencent Hy3 [44]. All configurations use the same input specifications, generation protocol, and runtime evaluation prompt.

The evaluator returns EXCELLENT, USABLE, or UNUSABLE. We report coverage, **Excellent rate** R_{exc} , and **Usable rate** R_{use} . Coverage is the fraction of evaluation attempts with a valid returned label. Artifact-attributable installation, build, serving, or access failures are valid UNUSABLE labels; invalid labels are limited to evaluator or deployment infrastructure failures, corrupted evidence packages, or unresolved evaluator timeouts that prevent reliable scoring. R_{exc} is the fraction of valid scored samples labeled EXCELLENT. R_{use} is the fraction of valid scored samples labeled EXCELLENT or USABLE, i.e., whether the artifact crosses the minimum playable-delivery threshold.

4.2 Model Capability Differentiation

Table 1 shows that WebGameBench produces clear capability separation among current coding agents. The strongest two configurations, ‘opus-4-7’ and ‘opus-4-6’, reach R_{use} values of 76.9% and 73.0%, while lower-scoring configurations fall in the 38.3–52.8% range. Middle configurations form a continuous band around 60%, indicating that the benchmark does not collapse all configurations into the same success or failure region, but instead provides interpretable model differences.

Usable rate R_{use} indicates whether an artifact is labeled EXCELLENT or USABLE, but it is not a high-quality completion metric. Even the best configuration still leaves roughly one quarter of tasks labeled UNUSABLE. More importantly, **Excellent rate** R_{exc} is much lower: the highest R_{exc} is only 20.2%, and most configurations are below 17%. This gap shows that crossing the minimum

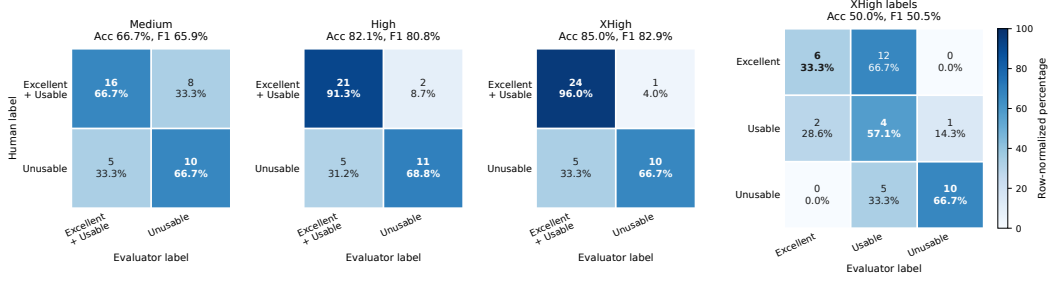


Figure 4: Agreement between the runtime evaluator and human-review labels. The first three heatmaps use the R_{use} label, merging EXCELLENT and USABLE into *Excellent + Usable*; the fourth keeps the three labels under XHigh to inspect the R_{exc} boundary. Colors are row-normalized by the human label, and cell text reports counts and row percentages.

playable-delivery threshold is substantially easier than satisfying the full set of complex runtime requirements.

The D1–D4 columns further show that the construction-time D -level is directionally aligned with runtime usable rate. Aggregating the usable/valid counts in Table 1 by D -level gives pooled usable rates of $123/167 = 73.7\%$, $220/289 = 76.1\%$, $494/948 = 52.1\%$, and $12/95 = 12.6\%$ for D1, D2, D3, and D4, respectively. Thus D1/D2 tasks remain near a 75% usable rate, D3 drops to about 52%, and D4 reaches only 12.6%. We therefore interpret D -level as a coarse risk stratification useful for explaining shared failures on D3/D4 tasks, rather than as a strict total order over individual task difficulty.

4.3 Agreement with Human Review

We evaluate whether the runtime evaluator agrees with independent human gameplay review on a 43-artifact human-review set covering Medium, High, and XHigh reasoning settings. Human review was performed by five annotators, with two independent human tests per artifact. Human labels follow a playable-lifecycle criterion: a game is usable if the player can enter or start it, exercise the core input, receive rule feedback, observe state or score updates, and reach a win/loss or ending state; failures to start or end, mid-game deadlocks, broken core input, and incorrect win/loss logic are marked UNUSABLE. Two senior experts then inspected disagreements between human labels and automatic evaluator outputs, focusing on whether they arise at the R_{use} boundary between EXCELLENT/USABLE and UNUSABLE, or at the R_{exc} boundary between EXCELLENT and USABLE.

Figure 4 reports agreement for the R_{use} label and for exact three-way labels under XHigh. Agreement on R_{use} improves with evaluator reasoning strength: Medium reaches 66.7% accuracy and 65.9% macro-F1, High reaches 82.1% and 80.8%, and XHigh reaches 85.0% and 82.9%, supporting the evaluator as a scalable signal for Usable-rate statistics. Exact three-way agreement remains lower under XHigh, at 50.0% accuracy and 50.5% macro-F1, mainly because the R_{exc} boundary is harder: 12 samples labeled EXCELLENT by humans are downgraded to USABLE by the evaluator. We therefore use automatic evaluation for aggregate Usable-rate statistics and runtime failure diagnosis, while treating Excellent-rate analysis as a stricter quality signal that still benefits from human review.

4.4 Runtime Case Studies

To connect aggregate labels to runtime evidence, we analyze paired traces from Kimi K2.5 and Claude Opus 4.6 on six games; Figure 5 shows the two contrast cases most directly tied to our main claim, with the remaining cases in Appendix F.

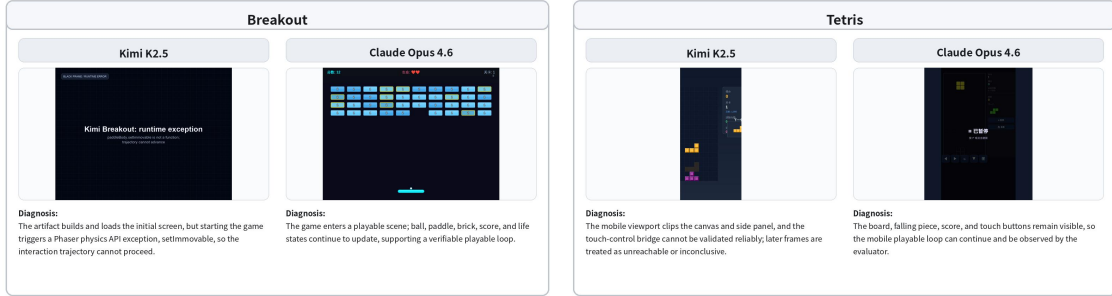


Figure 5: Two representative runtime case studies in the main text. Each task compares Kimi K2.5 and Claude Opus 4.6, with evaluator diagnosis shown below the corresponding screenshots.

Figure 5 illustrates why buildability and loadability are not application usability. In Breakout, the Kimi artifact loads but fails after start because a Phaser physics exception breaks the control loop, while the Opus artifact preserves ball, paddle, brick, score, and life updates. In Tetris, Kimi’s mobile layout and touch bridge prevent stable playable evidence, while Opus keeps the board, falling pieces, score, and controls observable. These cases show that runtime evaluation diagnoses interaction, state-transition, and rule-execution failures that build logs or screenshots can miss.

5 Limitations and Discussion

WebGameBench has several limitations. The runtime evaluator supports scalable Usable-rate estimation and failure diagnosis, but it is not a substitute for human review: agreement on the Usable-rate criterion is reasonable, while exact three-way quality labeling remains limited. The benchmark is instantiated with browser-native games, so its results should be read as evidence about browser-accessible interactive artifact delivery rather than arbitrary software-engineering competence. The frozen specifications are produced through fixed templates and filtering, and candidate precondition construction cannot exhaust long-horizon, randomized, or multi-session behavior. We therefore report aggregate scores together with structured evidence and human-review analysis, and position WebGameBench as a diagnostic complement to human review rather than a standalone certification of application quality.

6 Conclusion

We introduced WebGameBench, a requirement-to-application benchmark for coding agents based on browser-native games. Each task provides a frozen Structured WebGame Specification; the agent generates a source artifact; the artifact is built, served, and exposed as a browser-accessible URL under the same deployment protocol; and a runtime evaluator verifies input handling, spatial mapping, rule execution, state transitions, terminal conditions, and restart behavior through real browser interaction. Experiments on 111 tasks, 12 coding agents, and 14 evaluation configurations show that WebGameBench differentiates current models’ delivered-application capability in this setting: the best configuration reaches a 76.9% usable rate, but only a 20.2% excellent rate, showing that crossing the minimum playable-delivery threshold is not equivalent to complete requirement satisfaction. Further *D*-level stratification, human review, and runtime case studies show that WebGameBench supports aggregate scoring, Usable-rate human-alignment analysis, and runtime failure diagnosis. Overall, browser-native games provide a compact and behavior-dense testbed for evaluating whether coding agents can deliver runnable, interactive, and diagnosable software artifacts from structured requirements.

References

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke

- Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [2] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, 2025.
 - [3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.
 - [4] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
 - [5] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024.
 - [6] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
 - [7] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
 - [8] Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.
 - [9] Niels Mündler, Mark Niklas Müller, Jingxuan He, and Martin Vechev. Swt-bench: Testing and validating real-world bug-fixes with code agents. *Advances in Neural Information Processing Systems*, 2024.
 - [10] Ben Bogin, Kejuan Yang, Shashank Gupta, Kyle Richardson, Erin Bransom, Peter Clark, Ashish Sabharwal, and Tushar Khot. Super: Evaluating agents on setting up and executing tasks from research repositories. In *Proceedings of EMNLP*, 2024.
 - [11] Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal Patwardhan. Paperbench: Evaluating ai’s ability to replicate ai research. *arXiv preprint arXiv:2504.01848*, 2025.
 - [12] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. Mle-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024.
 - [13] Zimu Lu, Yunqiao Yang, Houxing Ren, Haotian Hou, Han Xiao, Ke Wang, Weikang Shi, Aojun Zhou, Mingjie Zhan, and Hongsheng Li. Webgen-bench: Evaluating llms on generating interactive and functional websites from scratch. *arXiv preprint arXiv:2505.03733*, 2025.

- [14] Chenxu Liu, Yingjie Fu, Wei Yang, Ying Zhang, and Tao Xie. Webcoderbench: Benchmarking web application generation with comprehensive and interpretable evaluation metrics. *arXiv preprint arXiv:2601.02430*, 2026.
- [15] Jingyao Liu, Chen Huang, Zhizhao Guan, Wenqiang Lei, and Yang Deng. E2edev: Benchmarking large language models in end-to-end software development task. *arXiv preprint arXiv:2510.14509*, 2025.
- [16] Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. Design2code: Benchmarking multimodal code generation for automated front-end engineering. In *Proceedings of NAACL*, 2025.
- [17] Ryan Li, Yanzhe Zhang, and Diyi Yang. Sketch2code: Evaluating vision-language models for interactive web design prototyping. In *Proceedings of NAACL*, 2025.
- [18] Jingyu Xiao, Yuxuan Wan, Yintong Huo, Zixin Wang, Xinyi Xu, Wenxuan Wang, Zhiyao Xu, Yuhang Wang, and Michael R. Lyu. Interaction2code: Benchmarking mllm-based interactive webpage code generation from interactive prototyping. In *Proceedings of ASE*, 2025.
- [19] Zhiyu Lin, Zhengda Zhou, Zhiyuan Zhao, Tianrui Wan, Yilun Ma, Junyu Gao, and Xuelong Li. Webuibench: A comprehensive benchmark for evaluating multimodal large language models in webui-to-code. In *Findings of ACL*, 2025.
- [20] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*, 2023.
- [21] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of ACL*, 2024.
- [22] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. Webvoyager: Building an end-to-end web agent with large multimodal models. In *Proceedings of ACL*, 2024.
- [23] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. Workarena: How capable are web agents at solving common knowledge work tasks? *arXiv preprint arXiv:2403.07718*, 2024.
- [24] Thibault Le Sellier De Chezelles, Maxime Gasse, Alexandre Drouin, Massimo Caccia, Léo Boisvert, Megh Thakkar, Tom Marty, Rim Assouel, Sahar Omidi Shayegan, Lawrence Keunho Jang, Xing Han Lù, Ori Yoran, Dehan Kong, Frank F. Xu, Siva Reddy, Quentin Cappart, Graham Neubig, Ruslan Salakhutdinov, Nicolas Chapados, and Alexandre Lacoste. The browsergym ecosystem for web agent research. *arXiv preprint arXiv:2412.05467*, 2024.
- [25] Christopher Rawles, Sarah Clinckemaulle, Yifan Chang, Jonathan Waltz, Gabrielle Lau, Marybeth Fair, Alice Li, William Bishop, Wei Li, Folawiyi Campbell-Ajala, Daniel Toyama, Robert Berry, Divya Tyamagundlu, Timothy Lillicrap, and Oriana Riva. Androidworld: A dynamic benchmarking environment for autonomous agents. *arXiv preprint arXiv:2405.14573*, 2024.
- [26] Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Ruoyu Tao, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. Textworld: A learning environment for text-based games. *arXiv preprint arXiv:1806.11532*, 2018.
- [27] Matthew J. Hausknecht, Prithviraj Ammanabrolu, Marc-Alexandre Côté, and Xingdi Yuan. Interactive fiction games: A colossal adventure. In *AAAI Conference on Artificial Intelligence*, 2019. URL <https://api.semanticscholar.org/CorpusID:202565447>.
- [28] Diego Pérez-Liébana, Jialin Liu, Ahmed Khalifa, Raluca D. Gaina, Julian Togelius, and Simon M. Lucas. General video game ai: A multitrack framework for evaluating agents, games, and content generation algorithms. *IEEE Transactions on Games*, 2019.

- [29] William H. Guss, Cayden Codel, Katja Hofmann, Brandon Houghton, Noboru Kuno, Stephanie Milani, Sharada Mohanty, Diego Perez Liebana, Ruslan Salakhutdinov, Nicholay Topin, Manuela Veloso, and Phillip Wang. The minerl 2019 competition on sample efficient reinforcement learning using human priors. In *Advances in Neural Information Processing Systems*, 2019.
- [30] Anthony Costarelli, Mat Allen, Roman Hauksson, Grace Sodunke, Suhas Hariharan, Carlson Cheng, Wenjie Li, Joshua Clymer, and Arjun Yadav. Gamebench: Evaluating strategic reasoning abilities of llm agents. *arXiv preprint arXiv:2406.06613*, 2024.
- [31] Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wolczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. Balrog: Benchmarking agentic llm and vlm reasoning on games. In *International Conference on Learning Representations*, 2025.
- [32] Lanxiang Hu, Mingjia Huo, Yuxuan Zhang, Haoyang Yu, Eric P. Xing, Ion Stoica, Tajana Rosing, Haojian Jin, and Hao Zhang. lmgame-bench: How good are llms at playing games? *arXiv preprint arXiv:2505.15146*, 2025.
- [33] Christopher Zhang Cui, Xingdi Yuan, Ziang Xiao, Prithviraj Ammanabrolu, and Marc-Alexandre Côté. Tales: Text adventure learning environment suite. *arXiv preprint arXiv:2504.14128*, 2025.
- [34] Anthropic. Claude opus 4.7. <https://www.anthropic.com/news/claude-opus-4-7>, 2026. Accessed: 2026-05-07.
- [35] Anthropic. Claude opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>, 2026. Accessed: 2026-05-07.
- [36] Anthropic. Claude sonnet 4.5. <https://www.anthropic.com/news/claude-sonnet-4-5>, 2025. Accessed: 2026-05-07.
- [37] OpenAI. Introducing gpt-5.5. <https://openai.com/index/introducing-gpt-5-5/>, 2026. Accessed: 2026-05-07.
- [38] Google Gemini Team. Gemini 3.1 pro: A smarter model for your most complex tasks. <https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/>, 2026. Accessed: 2026-05-07.
- [39] DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.
- [40] Moonshot AI. Kimi k2.6: Scaling agentic intelligence. <https://www.kimi.com/blog/kimi-k2-6>, 2026. Accessed: 2026-05-07.
- [41] Kimi Team, Tongtong Bai, Yifan Bai, Yiping Bao, S. H. Cai, et al. Kimi k2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- [42] GLM-5 Team, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, et al. Glm-5: From vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.
- [43] Z.AI. Glm-5.1. <https://docs.z.ai/guides/llm/glm-5.1>, 2026. Accessed: 2026-05-07.
- [44] Tencent. Tencent unveils hy3 preview; model enhances agent capabilities and real-world usability. <https://www.tencent.com/en-us/articles/2202320.html>, 2026. Accessed: 2026-05-07.

A Specification Construction Details

This appendix records construction details that are intentionally compressed in the main text. The main text defines the Structured WebGame Specification as the shared fixed requirement representation used in evaluation. Here we explain how upstream materials are abstracted into that task input. In the engineering pipeline, this frozen requirement document was sometimes called a PRD. In the paper, we consistently call it a Structured WebGame Specification: a task contract produced from an original user request or seed through a fixed template, and shared by both agent generation and runtime evaluation.

A.1 Seed Abstraction

WebGameBench uses public browser-game portals and user-style requests only as inspiration for gameplay families and interaction patterns. A candidate page or upstream user request is first reduced to a task seed: it preserves the playable concept while removing brand names, site-specific copy, advertisements, accounts, leaderboards, concrete art assets, audio, level files, implementation details, and external service dependencies. The seed is not the benchmark input. It is an intermediate construction artifact used to synthesize a self-contained Structured WebGame Specification.

A.2 Specification Template

Each Structured WebGame Specification follows a fixed product-requirements-style template:

- game overview: name, type, one-sentence description, and player objective;
- page and flow: initial page, game page, ending or settlement page, and restart;
- input controls: primary input, auxiliary input, and invalid-input feedback;
- game objects: player objects, non-player objects, board or scene, and random objects;
- rules and state changes: initial state, per-frame or per-turn changes, valid-action effects, score or resource changes, difficulty changes, failure state, and victory or level-clear state;
- visual feedback: layout, state display, operation feedback, success feedback, and failure feedback;
- technical and deployment constraints: running mode, external dependencies, storage, devices and browsers, and forbidden items;
- evaluable functional points: loading, start or restart, input response, core rules, observable state changes, score/resource/progress updates, terminal state, optional victory or level-clear state, and no dependence on private assets or remote services.

A.3 Specification Synthesis Prompt

The specification synthesis prompt asks a model to rewrite a short user request or task seed into a complete, implementable, and evaluable Structured WebGame Specification, rather than implementation code. The prompt uses a fixed section skeleton: application overview, users and usage scenarios when applicable, page structure and core functions, business rules and logic, exceptions and boundary cases, acceptance criteria, and out-of-scope functionality. For game tasks, the prompt further requires background and goals to serve gameplay description, core functions to center on the minimum playable loop, business rules to express gameplay rules and win/loss or ending conditions, and page or level structure to remain reasonably simple while preserving key exceptions and boundary cases.

The prompt enforces four constraints. First, it preserves the game identity, core gameplay, and all details already given in the user input. Second, it follows an MVP boundary: it only adds capabilities that are necessary for the core objective, and does not add accounts, leaderboards, payments, remote services, future plans, or other unrelated extensions. Third, it does not specify a concrete technology stack or implementation path. Fourth, after entering the rewrite stage, it does not ask clarification questions and directly outputs a frozen requirement document in Markdown. The generated document is the Structured WebGame Specification used in this paper. Functional-point annotation, difficulty annotation, agent generation, and runtime evaluation all use this frozen specification as input.

B Functional-Point Annotation

The main text uses functional points as derived annotations for observable requirement decomposition and corpus profiling. This appendix gives the B-S-T functional-point schema, the atomic functional-point construction rules, and corpus-level statistics.

B.1 B-S-T Functional-Point Schema

Functional-point annotation starts from three observable requirement-decomposition axes and organizes them in a four-tier schema. In Tier 1, *B* denotes behavior, describing player entry, input actions,

and core interaction; S denotes spatial structure, describing position representation, coordinate mapping, boards, canvas regions, hit zones, and collision relations; and T denotes temporal/state behavior, describing game entities, phases, random or hidden information, resource changes, terminal conditions, state transitions, and temporal dependencies. Tier 2 defines stable subdimensions under each Tier-1 axis; Tier 3 defines controlled tags or enumerated values under each subdimension; and Tier 4 is the lowest-level task-specific functional point, namely an independently observable atomic behavior or structural assertion.

Tier	Meaning	Examples
Tier 1	Broad decomposition axis	B, S, T
Tier 2	Stable subdimension	B : entry/controls and interaction; S : topology and position precision; T : layout/display, entities/resources, phases/transitions, hidden/random information, and temporal horizon
Tier 3	Controlled tag or value	entry, input response, spatial mapping, collision, state transition, terminal condition
Tier 4	Atomic functional point	“Pressing the left arrow moves the player left”; “The game shows a restart control after failure”

B.2 Atomic Functional-Point Rules

An atomic functional point is the counting unit used in corpus profiling. Each item must be atomic, independently observable, and grounded in the frozen Structured WebGame Specification. If one requirement contains multiple observable behaviors, it is decomposed into multiple atomic functional points. For example, a four-direction movement requirement can be split into separate items for moving up, down, left, and right; a complete terminal flow can be split into failure detection, settlement display, and restart entry.

Each atomic functional point is associated with one axis/subdimension/tag path and bound to a concrete UI or interaction component. If the assertion describes overall topology or global state behavior, it is bound to the global game structure. Pure visual polish is not counted as an independent atomic functional point unless the specification explicitly makes it an observable success condition. This keeps functional points focused on behavior, spatial mapping, state change, and user-visible feedback rather than implementation style.

B.3 Corpus Statistics

Using B-S-T functional-point annotation, the final 111 tasks contain 19–80 atomic functional points per task, with mean 36.20 and median 33. B-S-T annotation is used only for decomposition and corpus profiling, not for assigning the difficulty label. The D -level is determined separately by the $S_{\text{struct}}\text{-}L_{\text{logic}}$ lookup rule in Appendix C. When grouped by D -level, functional-point count increases on average, which serves as a corpus-level sanity check rather than a labeling rule.

D -level	N	Min	Max	Mean	Median
D1	12	20	36	25.67	25.5
D2	21	20	46	28.52	28
D3	71	19	80	38.46	36
D4	7	42	80	54.29	50

C Difficulty Annotation

The main text uses D -level as a coarse difficulty tag for experimental stratification and corpus profiling. This appendix gives the $S_{\text{struct}}\text{-}L_{\text{logic}}$ rubric, lookup rule, and stability check used to determine D -level.

C.1 Structural Scale and Logical Depth

Difficulty annotation first assigns two axes independently from the frozen Structured WebGame Specification. S_{struct} measures UI and navigation scale: S1 denotes 1–2 pages, flat structure, and no

nesting; S2 denotes 3–5 pages, linear navigation, and standard component stacking; S3 denotes 5–10 pages with secondary nesting, global component extraction, or region partitioning, and also includes applications with more than 10 pages but shallow module hierarchy; S4 denotes more than 10 pages with deep module hierarchy or multiple independent role-specific portals. L_{logic} measures rule and acceptance depth: L1 denotes static or nearly static presentation, no substantive state management, and fewer than 5 acceptance criteria; L2 denotes form validation, CRUD, persistence, or simple linear state transitions, with roughly 5–15 acceptance criteria; L3 denotes more than 15 acceptance criteria, complex state machines, conditional branches or rollback paths, or cross-module consistency constraints; L4 denotes complex algorithms, core rule engines, strong consistency, or multi-session real-time synchronization.

C.2 D-level Lookup Rule

After the two axes are assigned, D -level is obtained by a fixed lookup table:

$S_{\text{struct}}/L_{\text{logic}}$	L1	L2	L3	L4
S1	D1	D1	D3	D3
S2	D1	D2	D3	D3
S3	D2	D2	D3	D4
S4	D2	D3	D4	D4

The lookup rule makes D -level deterministic once S_{struct} and L_{logic} are determined, and intentionally makes logical depth dominant. Thus compact tasks with L3/L4 logic enter at least D3, while multi-page tasks with standard L2 logic can remain D2.

C.3 Operational Annotation and Stability

The annotation workflow first extracts evidence for page count, module hierarchy, and user-role-specific interface scope from the specification’s page-structure and core-function sections, and assigns S_{struct} . It then extracts evidence for rule count, exception branches, state machines, and acceptance-criteria count from the rules, exceptions, boundary cases, and acceptance sections, and assigns L_{logic} . Finally, it strictly applies the lookup rule to output the D -level and a one-sentence implementation-risk summary. The final prompt explicitly covers all 16 $S_{\text{struct}}-L_{\text{logic}}$ combinations and adds boundary rules to reduce ambiguity for S1+L2, S2+L3, S3+L2, and related cases. We check stability through repeated annotation and cross-prompt-version comparison. On the benchmark data, the final version reaches approximately 90% single-request D -level agreement against majority labels from repeated runs.

C.4 Difficulty Annotation Prompt

The final difficulty annotation prompt is:

You are a senior software architect responsible for quantitative complexity grading of a Structured WebGame Specification. Analyze the breadth of UI structure S and the depth of business logic L , and determine the final difficulty D . Larger numbers indicate higher implementation risk.

S structural scale. S1: 1–2 pages, flat structure, no nesting. S2: 3–5 pages, linear navigation, standard component stacking. S3: either 5–10 pages with secondary nesting, global component extraction, or region partitioning; or more than 10 pages with shallow module hierarchy. S4: more than 10 pages and either deep module hierarchy or multiple independent role-specific portals.

L logical depth. L1: static or nearly static presentation, no substantive state management, fewer than 5 acceptance criteria. L2: form validation, CRUD, persistence, or simple linear state transition, usually 5–15 acceptance criteria. L3: more than 15 acceptance criteria, or complex state machines, conditional branches, rollback paths, or cross-module consistency constraints. L4: complex algorithms, core rule engines, strong consistency, or multi-session real-time synchronization.

First determine S and L independently from the frozen specification, then obtain D strictly from the lookup rule in Appendix C; do not infer it subjectively. Boundary rules: when $L \geq L3$, D is at least D3; S1+L2 is D1; S1/S2+L3 is D3; S3/S4+L2 is D2; D4 appears only

for S3+L4 or S4+L3/L4. Do not use model generation results, runtime evaluator results, or B-S-T atomic functional-point counts to determine D . Output only three lines: S: {S1-S4} | evidence, L: {L1-L4} | evidence, and D: {D1-D4} | lookup evidence.

D Runtime Evaluator Prompt and Report Schema

The main text defines the runtime evaluation protocol. This appendix records the evaluator’s operational prompt schema. Main experiments use Codex CLI 0.115.0 with the ‘gpt-5.4-agent’ backend at XHigh reasoning effort, a two-hour wall-clock timeout per evaluation rollout, and no additional fixed browser-action step cap. The human-review agreement analysis reruns the same protocol under Medium, High, and XHigh reasoning settings.

D.1 Evaluator Inputs

Each evaluation receives:

- a browser-accessible deployed URL;
- the frozen Structured WebGame Specification for the same task;
- optional source bundle, build logs, serving logs, browser logs, screenshots, traces, and generation metadata for diagnosis.

The evaluator does not receive a reference implementation and does not provide feedback to the same agent attempt.

D.2 Interaction Rules

The evaluator prompt enforces the following order:

1. open the deployed URL with Playwright and handle browser-access issues when needed;
2. perform a short smoke interaction to check loadability, entry, and the main playable state;
3. derive checks from the frozen specification and generic playable-loop requirements;
4. verify checks through user-level actions before relying on source code or logs;
5. record the pre-trigger state, trigger action, visible result, and relevant state or numeric deltas;
6. mark each acceptance item as passed, failed, or unverified.

Source code may explain a failure, but it cannot by itself prove that a user-visible behavior works.

D.3 Candidate Preconditions

For long-horizon, rare, or randomized states, code or runtime-state manipulation may be used to construct a candidate precondition. This operation does not itself verify the target function. The evaluator must first confirm that the candidate state is visible on the page or stably readable from runtime state, and then execute the final step through real user interaction. If the candidate state cannot be confirmed, the item is recorded as failed to construct, unstable to reproduce, or requiring review, rather than directly counted as a functional failure.

D.4 Spatial and Multi-Session Rules

Spatial checks must compare the intended operation location with the actual effect location. For boards, grids, canvas regions, drag slots, hit zones, placement cells, and connection points, a reaction is insufficient: the visible operation point and the effective point must agree. If visual interaction conflicts with DOM or state inspection, the evaluator prioritizes the result closer to normal user operation.

Multi-session checks are enabled only when required by the specification. The evaluator must construct independent sessions, record the visible state in each session, and check room creation, joining, start conditions, turn/state synchronization, settlement consistency, exit, refresh, and ownership behavior according to the requirement. A single endpoint cannot prove a multi-session mechanism.

D.5 Report Schema

Each report contains:

1. final conclusion: row or sample id, deployed URL, source reference, quality label, and one-sentence summary;
2. core functional checks: pass, fail, or unverified for the main playable loop;
3. other issues: non-core failures and whether they affect the final label;
4. acceptance results: per-requirement observations and evidence;
5. unverified items: reason, attempted interactions, and whether review is needed.

E Compute Resources and Trace Statistics

All generation and evaluation jobs are executed through API calls on a CPU cluster. We do not train models or perform GPU fine-tuning. Agent generation uses concurrency 6, and runtime evaluation uses concurrency 20; each concurrent job corresponds to one independent CPU-cluster execution unit. The per-sample timeout is 2 hours for both generation and evaluation. Because different coding-agent APIs have different service-side implementations, latency, and pricing, we do not report exact per-model dollar cost. Instead, we use turn counts from generation and evaluation traces as a compute-effort proxy.

Table 2 reports available traces for 111 game tasks in the WebGameBench benchmark data under selected model configurations. The table is used to confirm the data status and scale of API batch runs, not as full per-model cost accounting. Each row corresponds to one coding-agent configuration evaluated on the same benchmark tasks. The Opus 4.6 row corresponds to the ‘opus-4-6’ generation-model configuration in the main experiment. Generation turns count assistant response turns in the generation trajectory; evaluation turns count assistant/agent messages in the runtime evaluator rollout. This table reports only sampled configurations used for data inspection and does not replace the full model comparison in Table 1. Game N is the number of benchmark game tasks for that configuration; Gen N and Eval N are the numbers of samples with parsed generation trajectories and evaluation rollouts; mean, med., P90, and max denote the mean, median, 90th percentile, and maximum turn counts per sample.

Table 2: Generation/evaluation trace turn statistics for selected model configurations on WebGameBench benchmark data. P90 denotes the 90th-percentile turn count.

Model	Game N	Gen N	Gen mean	Gen med.	Gen P90	Gen max	Eval N	Eval mean	Eval med.	Eval P90 / max
Opus 4.6	111	111	20.3	19	36	47	100	34.8	34.0	49 / 75
Sonnet 4.5	111	111	22.1	19	39	63	102	33.9	32.5	49 / 63
Kimi K2.5	111	111	23.7	20	45	66	107	31.2	29.0	44 / 68

Overall, the three inspected configurations all have complete generation traces, with average generation turns around 20–24. Parsed evaluation rollouts cover 100–107 game tasks, with average evaluation turns around 31–35. These results indicate that the main experimental batch runs are dominated by bounded API generation and browser runtime evaluation, with interaction scale in the tens of turns, rather than by manual long-horizon debugging or open-ended repeated attempts.

F Supplementary Runtime Case Studies

The main text keeps only the Breakout and Tetris contrast cases in Figure 5. This section supplements the remaining four tasks from case-study v3, covering boundary cases that are not expanded in the main text: entry failure with partially verifiable core rules, positive playable traces, dependency blockage, and missing evidence.

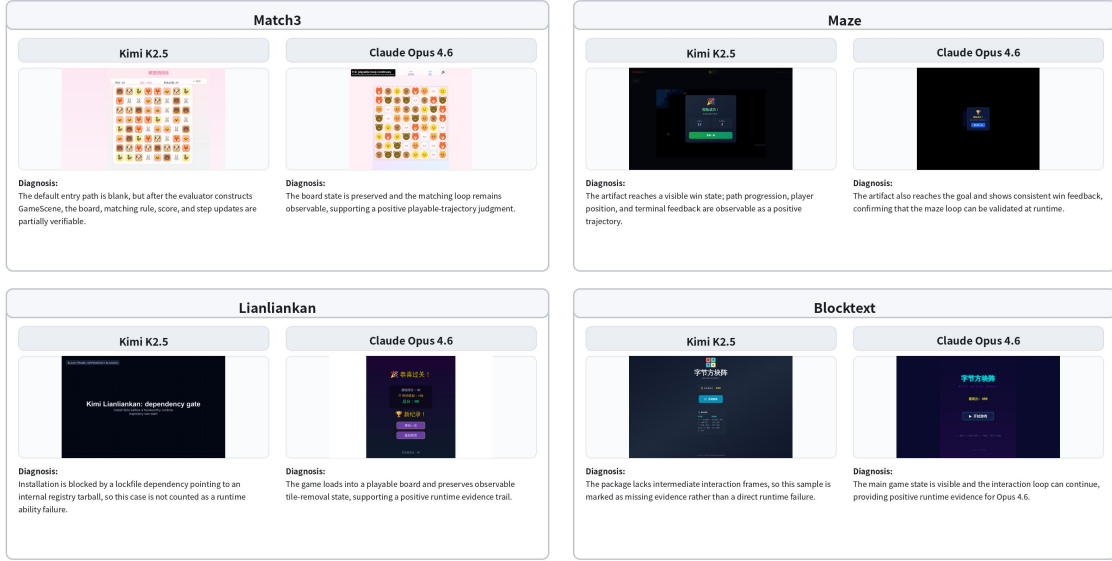


Figure 6: Supplementary runtime case studies for the four tasks omitted from the main text. Each task compares Kimi K2.5 and Claude Opus 4.6 with one representative screenshot and an evaluator diagnosis.

Match3. Kimi K2.5’s default entry path is blank, but after the evaluator constructs entry into GameScene, the board, matching rules, score, and step updates can be partially verified. This sample should therefore not be treated as a pure blank-screen failure; it is better recorded as mixed evidence: entry failure with partially running core rules. Claude Opus 4.6 preserves the full board state, and the elimination loop remains observable.

Maze. Both Kimi K2.5 and Claude Opus 4.6 can reach a visible victory state. This case serves mainly as a positive control: the runtime evaluator records not only failures, but can also follow the requirement-to-observation chain to verify path progress, player-position changes, and terminal feedback.

Lianliankan. Kimi K2.5 is blocked during installation because its lockfile points to a private registry tarball. Under the scoring rule in Section 4, this is a valid UNUSABLE delivery failure because the artifact cannot be entered by a user; in the case analysis, we separate it from browser-runtime interaction failures. Claude Opus 4.6 can load a playable board and preserves observable tile-removal state.

Blocktext. Kimi K2.5’s evidence package lacks intermediate interaction frames and retains only the menu or terminal screen. The more appropriate conclusion is therefore evidence missing, rather than direct runtime failure. Claude Opus 4.6 shows a visible main interface and game state, and its interaction loop can continue.

G Case Evidence for Runtime Evaluation

The following cases illustrate the requirement-to-observation chain. They are not additional aggregate metrics.

Tetris-like game. Human review noted that combo scoring and level progression are hard to verify through short natural play. The evaluator constructs candidate states for single-line clearing, four-line clearing, near-threshold score, spawn collision, and different tetromino types. After confirming each state, it triggers the final action through keyboard input. This process verifies line clearing and combo scoring, but exposes a level-progression bug: reaching the target score does not advance the level.

Farm management game. The task contains multi-year decisions, resource constraints, random events, insufficient funds, and early termination under low sustainability. The evaluator constructs candidate states for insufficient funds, year-5 settlement, and low sustainability, and then submits the final real action. The main loop is playable, but the page lacks crop growth-state feedback and implements a technology event inconsistently with the specification.

Gold Miner game. Natural play shows that the hook is too fast for reliable progress. The evaluator first verifies hook swing, capture, item value, and retrieval through real clicks, then constructs a level-clear candidate state to inspect the shop chain. Even after reaching the target score, the game still enters failure when the countdown ends; purchased power-ups also do not persist to the next level.

UNO-style game. UNO prompts, draw behavior, function cards, and terminal conditions require specific hands. The evaluator uses natural play and candidate hands, then clicks visible cards or the draw pile. It finds that illegal plays can be accepted, emptying the player's hand does not immediately trigger settlement, and the visible draw-pile region is blocked by a decorative layer.